



Open Heterogenous Runtime for Quantum Computing & Classical Accelerators

Muhammad Osama, Yasuko Ecker, Xulong Tang
AMD Research & Advanced Development

In Collaborations with Amir Shehata (ORNL)





*Key observation is that there is
no standard device runtime
and task interface shared
across quantum and HPC
stacks.*

Existing abstractions common abstractions.

CUDACUDA Driver and
CUDA Runtime APIs.

- Manually manage contexts, modules, streams and launch kernels; gives you “full” control at the driver-layer.
- Runtime-layer hides complexity behind <<<grid, block, shmem, stream>>> syntax,
- Stream-centric, concurrency is expressed through streams (kernels, memcpy, events),
- Graph capture, records streams to build DAG and replays it.
- Synchronize, streams, device, events, and `grid.sync()` (in cooperative-mode).

HIP

HIP Runtime APIs

- Inherits CUDA’s **stream-centric model** (even though the underlying hardware, AMD’s AQL command processor is a queue-based user-mode ring buffer.)
- Builds on top of HSA on AMD hardware, every stream wraps an AQL queue underneath – you never interact with AQL packets or HSA agents directly.
- HIP **Graph capture**, can’t control individual AQL packet fields, barrier bits, or signal semantics within the graph.

Existing abstractions common abstractions.

CUDA



CUDA Driver and
CUDA Runtime APIs.

HIP



HIP Runtime APIs

***Key abstraction, think in streams
and events. The hardware command
processor, ring buffers, signal
mechanisms, placement, do not exist.***

Existing abstractions common abstractions.

- **Agent-based:** System is a collection of agents (CPUs, GPUs, DSPs). Enumerate them, discover capabilities and memory pools.
- **User-mode queues:** Memory-mapped ring buffer. AQL packets directly write to buffer and writes to a doorbell to register to notify the hardware.
- **AQL Packets:** Strict to write into to a shared ring buffer that tells the GPU “run this kernel with these settings.”
- **Signals:** An atomic 64-bit integer the GPU decrements when work finishes, and the CPU waits on to know it’s done.

HSA



HSA Runtime – Native HW abstraction for AMD GPUs.

1. Create signal (value = 1)
2. Write 64 bytes into queue slot:
"run kernel X, args at ptr Y,
grid=1024, block=64, signal=this one"
3. Poke doorbell
4. Wait for signal == 0
5. Done. Read your results.



*What are the right set of knobs
required for an open
heterogenous runtime for
QHPC workloads?*

How we're thinking about the heterogenous device-runtime.

1

target - Device discovery and compilation for the right architecture.

2

device – **Where** (device, Partition, Sub-partition) to assign the work?

3

dispatch – **Mechanisms (what strategy)** to schedule the work; graphs, batch, etc.

4

schedule – **How** to place/schedule the work/tasks?

5

cooperative – **What** work runs concurrently/cooperatively?

6

memory – **Where and how** is the memory allocated, transferred, shared?

Example of why these knobs are important.

3

dispatch – Mechanisms (what strategy) to schedule the work;
graphs, batch, etc.

Latency Focused

Real-time Decode in Critical Path,
application needs to route quantum work
through a classical decode step before the
next quantum layer executes.

Bandwidth or Throughput Focused

Hybrid variational loop
(CPU/GPU/FPGA/QPU), application needs
to express dependency graph across
heterogenous devices and receive results
with near perfect load-balancing.



1. Runtime interface and API specification v0.1. Define the vendor-neutral runtime interfaces and APIs based on the agreed conceptual runtime object model.



2. Reference prototype. An open-source runtime prototype extended with a QPU adapter running a heterogeneous CPU/GPU/FPGA/QPU runtime graph end to end.





1. How does the runtime assign a task to a specific device?
2. What is the minimum runtime scheduler interface?
3. What are the right set of knobs a device-runtime should define?
4. How do these knobs are managed across heterogenous devices (do all devices equally provide functionality for all knobs?)